Distributed Live Chat and Reaction Aggregation System

Sicheng Xue (Eric) • Yun Li • Yumeng Zeng(Molly)

The Problem

Live Streaming Challenges

- Twitch, YouTube Live, Zoom webinars
- Thousands join when stream starts
- Everyone chats and reacts at once
- Traffic spikes are sudden and huge

Key Challenges

- Hot partitions: one viral room
- High write throughput needed
- Database can become bottleneck
- Must handle failures gracefully

Our Solution

Chat API

POST /messages
Submit new chat message

GET /messages
Fetch recent messages

Reaction API

POST /reactions
Submit reaction event

GET /reactions
Read aggregated counts

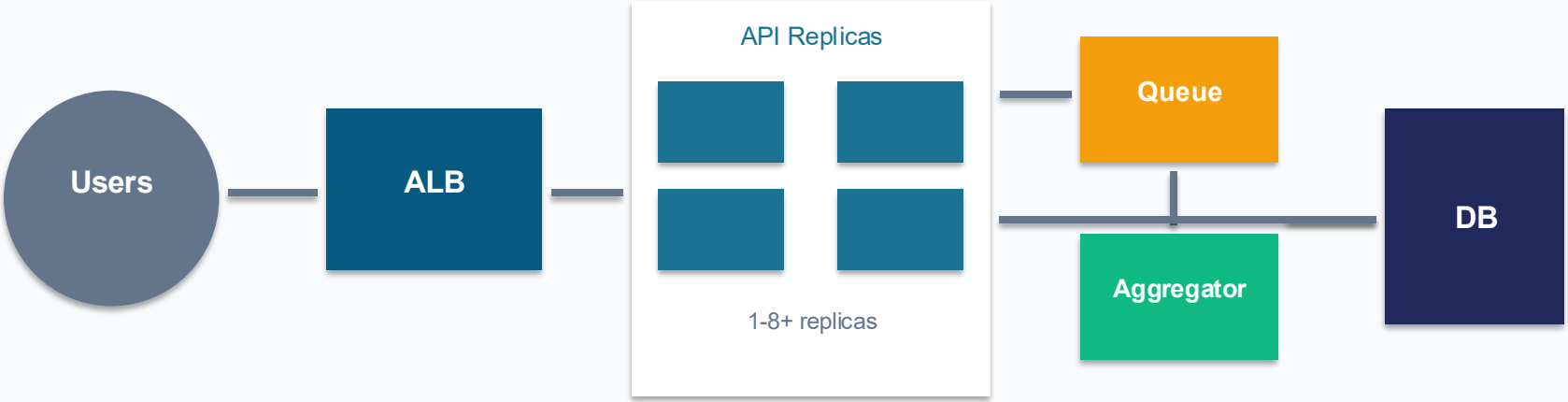
Infrastructure

AWS + Terraform
ECS + ALB deployment

Monitoring
CloudWatch metrics

Built with Go • Deployed on AWS • Tested with Locust

System Architecture



Chat: direct to DB | Reactions: Queue → Batch Aggregation → DB

Scaling Strategy

Horizontal Scaling

Stateless APIs behind ALB. Scale from 1 to 8+ replicas easily.

Hot Partition Handling

One viral room won't crash the system. Traffic distributed across nodes.

High Throughput Writes

Reactions go to queue first. Batch updates reduce DB pressure.

Fault Tolerance

If aggregator dies, events stay in queue. It catches up after recovery.

Experiment 1: Scale-Out Test

Setup

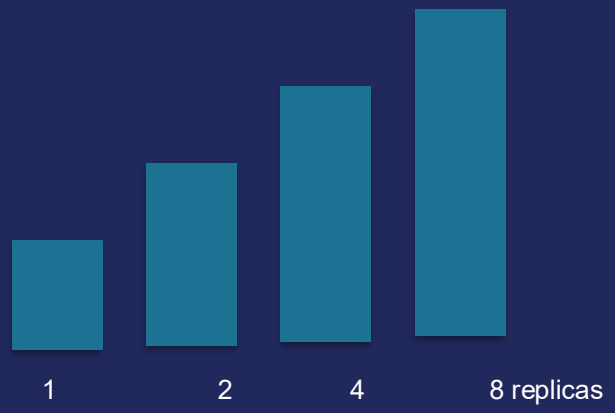
- Deploy 1 → 2 → 4 → 8 API replicas
- Keep load pattern constant
- All replicas behind one ALB

Metrics

- Throughput (requests/sec)
- p95 and p99 latency
- Error rate

Expected Result

Throughput scales linearly until bottleneck shifts to queue or storage.



Experiment 2: Hot Room vs Multi-Room

- Compare a hot-room workload with a multi-room workload
- In Workload A, 90% of the traffic goes to a single room
- This setting simulates a viral stream scenario
- In Workload B, traffic is spread evenly across many rooms
- This setting represents a more normal traffic pattern
- Measure CPU utilization, queue lag, and p95 and p99 latency
- This experiment studies the effect of traffic skew and hot partitions
- The hot-room workload is expected to place more pressure on the system
- Worse tail latency and more queue buildup are expected in Workload A

Workload A: Hot Room

90% of traffic goes to ONE room. Simulates viral stream scenario.

Workload B: Spread Traffic

Traffic spread evenly across many rooms. Normal usage pattern.

Experiment 3: Sync vs Event-Driven

Baseline: Synchronous

- Every reaction hits DB directly
- Simple but doesn't scale
- DB becomes bottleneck fast

VS

Improved: Event-Driven

- Reaction → Queue → Aggregator
- Batch updates to database
- Much higher throughput

Metrics: DB write rate, throughput, p99 latency

Goal: Prove decoupling reduces storage load and improves throughput

Why This Matters

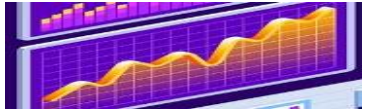
Live chat + reactions is a classic distributed systems challenge

Social Feeds

Twitter, Instagram, TikTok all face similar scaling problems.

Live Dashboards

Real-time analytics need the same patterns.



Metrics Pipelines

High-volume event processing everywhere.

Summary

- Go backend with Chat API + Reaction API
- Event-driven aggregation for high throughput
- Three experiments to test scaling behavior

Thank You! **Questions?**